Content Exclusions: Scoping, Precedence & IDE Behavior

Question: A security engineer configures a GitHub Copilot Content Exclusion rule at the organization level to exclude `**/secrets/**` and `*.pem`. A repository administrator then attempts to remove those patterns from their repository's Content Exclusion settings so developers can get inline suggestions inside `certs/prod.pem`. What is the resulting behavior?

A. The repository-level rule overrides the organization-level exclusion because repository settings are more specific.

B. Content exclusions are additive; repository administrators can add new exclusions, but cannot remove or weaken organization-level exclusions. **[CORRECT ANSWER]**

C. Developers can still trigger Copilot Chat on `certs/prod.pem` using `#file`, though inline ghost text remains disabled.

D. Content exclusions only take effect after adding a `.copilotignore` file to the repository root.

Explanation & Exam Rationale

GitHub Copilot **Content Exclusions** allow repository and organization administrators to specify `fnmatch` path patterns whose contents must never be read by Copilot to inform inline completions or Copilot Chat responses. Exclusions are configured in GitHub repository or organization settings (not via a local ignore file) and combine additively across hierarchy levels.

Aspect	Behavior / Rule	Exam Detail
Policy Hierarchy	Organization + Repository exclusions combine (union)	A repo admin can add stricter paths, never remove org rules
Inline Completions	Disabled inside excluded files	Copilot status icon shows excluded state in the IDE
Copilot Chat & `#file`	Excluded files are omitted from context	Explicitly attaching an excluded file with `#file` is blocked
Propagation Delay	Up to 30 minutes to sync to active IDEs	IDE reloads exclusion rules periodically or on window reload

Key Exam Takeaway: Content exclusions prevent files from being used as *context* for Copilot, and organization-level rules strictly enforce a security baseline that repository admins cannot override.